

CO3-AT1-Assignment

View Only

[← Back](#)**QUESTIONS****Q1**Banking Transaction System –
Exception Handling

10 marks

**Q2**Hospital Patient Registration –
User-Defined Exception

10 marks

**Q3**E-Commerce Order Processing
– Built-in Exceptions

10 marks

**Q4**Online Food Delivery – Multiple
Threads and Execution Order

10 marks

**Q5**Manufacturing Plant –
Producer-Consumer Problem**Q4 Online Food Delivery – Multiple Threads and Execution Order**

10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution

10 marks

5/5 answered

Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design
Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```
1 import java.util.Scanner;
2
3 class OnlineFoodDelivery{
4     static boolean paymentSuccess = false;
5
6     static class OrderThread extends Thread{
7         String order;
8
9         OrderThread(String order){
10             this.order=order;
11         }
12         public void run(){
13             System.out.println("Order Processing :")
14         }
15     }
16     static class PaymentThread extends Thread{
17         String payment;
18
19         PaymentThread(String payment){
20             this.payment = payment;
21         }
22         public void run(){
23             if(payment.equalsIgnoreCase("failed")){
24                 paymentSuccess=false;
25                 System.out.println("Payment failed")
26             }else{
27                 paymentSuccess = true;
```

Input

pizza UPI AgentA

Output

Order Processing :pizza
Payment successful:UPI
Delivery assinged :
AgentA

```
28         System.out.println("Payment success
29     }
30 }
31 }
32 static class DeliveryThread extends Thread {
33     String delivery;
34
35     DeliveryThread(String delivery) {
36         this.delivery = delivery;
37     }
38     public void run() {
39         if(paymentSuccess) {
40             System.out.println("Delivery assingn
41         }else{
42             System.out.println("Delivey assigne
43         }
44     }
45 }
46
47 public static void main(String[] args) throws
48     Exception{
49
50     Scanner sc = new Scanner(System.in);
51
52     String order = sc.next();
53     String payment = sc.next();
54     String delivery = sc.next();
55
56     OrderThread orderThread = new OrderThread(o
57     PaymentThread paymentThread = new PaymentTh
58     DeliveryThread deliveryThread = new Deliver
59
60     orderThread.start();
```

```
61         orderThread.join();
62
63         paymentThread.start();
64         paymentThread.join();
65
66         deliveryThread.start();
67         deliveryThread.join();
68     }
69 }
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.